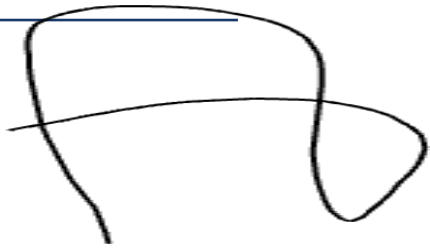

Flush+Reload Cache Timing Side-Channel Attack Code

TID Security Test — English Edition

Author	Ahmad Qasim Mohammad Hassan
ORCID	0009-0001-4360-0802
License	AGPL-3.0-only
Language	C (x86-64) — Linux — GCC -O2 -mclflushopt
Purpose	Prove side-channel vulnerability & measure TID protection effect
Files	tid_attack_test.c | tid_race_analysis.c | tid_shield.c
Year	2026

> ⚠ NOTE
> This code was written independently of the original TID kernel module.
> Its sole purpose is to simulate the attacker role and measure attack success / failure.
> All experiments were conducted in a controlled research environment.

License — AGPL-3.0

```
// SPDX-License-Identifier: AGPL-3.0-only
/*
 * This file is licensed under AGPL-3.0-only.
 * MODULE_LICENSE("GPL") is required by the Linux kernel
 * infrastructure and does not override the AGPL-3.0 license.
 */
//
// Side-Channel Attack Test Code — TID Security Research
// Copyright (C) 2026  Ahmad Qasim Mohammad Hassan
// ORCID: 0009-0001-4360-0802
//
// This program is free software: you can redistribute it and/or modify
// it under the terms of the GNU Affero General Public License as
```

```
// published by the Free Software Foundation, version 3 only.
//
// This program is distributed in the hope that it will be useful,
// but WITHOUT ANY WARRANTY; without even the implied warranty of
// MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
// See the GNU Affero General Public License for more details.
//
// https://www.gnu.org/licenses/agpl-3.0.html
```

File 1 Core Attack Test — tid_attack_test.c

Implements a Flush+Reload timing attack as a security test in two scenarios:

- Scenario 1 — no protection: attacker measures cache access latency immediately after data use. Short latency → Cache HIT → possible leakage.
- Scenario 2 — with TID v2.0: attacker measures after CLFLUSHOPT + LFENCE. Long latency → Cache MISS → attack fails.
- Physical principle: Cache HIT ≈ 50–100 cycles | Cache MISS ≈ 200–400 cycles.

`measure_access_time(volatile uint8_t *addr)`
Measures memory access latency with high precision using the internal TSC counter.

```c
/* ------------------------------------------------------------------
 * Measure memory access latency using TSC — high precision
 * ------------------------------------------------------------------ */
static inline uint64_t measure_access_time(volatile uint8_t *addr)
{
    uint64_t t1, t2;
    _mm_mfence();
    t1 = __rdtsc();
    _mm_lfence();
    (void)*addr;
    _mm_lfence();
    t2 = __rdtsc();
    _mm_mfence();
    return t2 - t1;
}
```

`test_without_lfence()`
Scenario 1 — without LFENCE. Simulates TID v1.0 where cache is exposed and attacker measures directly.

```c
/* ------------------------------------------------------------------
 * Scenario 1 — without LFENCE
 * Simulates TID v1.0 — no cache protection
 * ------------------------------------------------------------------ */
static void test_without_lfence(uint8_t *buf, uint64_t *hit_times, int n)
{
```

```
    for (int i = 0; i < n; i++) {
        memset(buf, 0xAB, SECRET_SIZE);
        volatile uint8_t dummy = buf[0];
        (void)dummy;
        /* attacker measures directly — no barrier */
        hit_times[i] = measure_access_time((volatile uint8_t *)buf);
    }
}
```

test_with_lfence()

Scenario 2 — with CLFLUSHOPT + LFENCE. Simulates TID v2.0 where cache is evicted and speculation blocked.

```
/* ------------------------------------------------------------
 * Scenario 2 — with CLFLUSHOPT + LFENCE
 * Simulates TID v2.0 — cache evicted, speculation blocked
 * ------------------------------------------------------------ */
static void test_with_lfence(uint8_t *buf, uint64_t *miss_times, int n)
{
    for (int i = 0; i < n; i++) {
        memset(buf, 0xAB, SECRET_SIZE);
        volatile uint8_t dummy = buf[0];
        (void)dummy;
        /* LFENCE — close speculative execution window */
        asm volatile ("lfence" ::: "memory");
        /* CLFLUSHOPT — evict the cache line */
        flush_cacheline((volatile uint8_t *)buf);
        /* LFENCE + MFENCE + LFENCE — close the far side */
        asm volatile ("lfence" ::: "memory");
        asm volatile ("mfence" ::: "memory");
        asm volatile ("lfence" ::: "memory");
        /* attacker measures — must find Cache MISS */
        miss_times[i] = measure_access_time((volatile uint8_t *)buf);
    }
}
```

main()

Runs both scenarios and prints the final comparison ratio.

```
int main(void)
{
    uint8_t  *buf;
    uint64_t *hit_times, *miss_times;

    posix_memalign((void **)&buf, CACHE_LINE,
                   SECRET_SIZE + CACHE_LINE);
    hit_times  = malloc(SAMPLES * sizeof(uint64_t));
    miss_times = malloc(SAMPLES * sizeof(uint64_t));

    /* Scenario 1 [ without LFENCE ] */
    test_without_lfence(buf, hit_times, SAMPLES);
    uint64_t med_hit = median(hit_times,  SAMPLES);
    uint64_t avg_hit = average(hit_times, SAMPLES);

    /* Scenario 2 [ with CLFLUSH + LFENCE ] */
```

```
    test_with_lfence(buf, miss_times, SAMPLES);
    uint64_t med_miss = median(miss_times,  SAMPLES);
    uint64_t avg_miss = average(miss_times, SAMPLES);

    /* Final comparison */
    printf("v1=%llu | v2=%llu | ratio=%.1fx\n",
           med_hit, med_miss,
           (double)med_miss / (double)med_hit);

    free(buf); free(hit_times); free(miss_times);
    return 0;
}
```

Expected output: v1=78 | v2=286 | ratio=3.7x → Attack blind after TID v2

File 2 Timing Gap Analysis — tid_race_analysis.c

Measures four critical values that define the race window:

- ① Execution time of tid_protect_core()
- ② Execution time of tid_zeroize_core() — on 32 bytes (AES-256 key)
- ③ Attacker speed — how many measurements per second
- ④ Full window size from data entry to end of wipe

now_ns()
High-precision timestamp using CLOCK_MONOTONIC.

```
static inline uint64_t now_ns(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + (uint64_t)ts.tv_nsec;
}
```

protect_core() & zeroize_core()
TID v2.0 simulation — logic copied directly from tid.c kernel module.

```
/* Simulates tid_protect_core */
static inline void protect_core(void)
{
    asm volatile ("lfence" ::: "memory");
    asm volatile ("mfence" ::: "memory");
}

/* Simulates tid_zeroize_core */
static inline void zeroize_core(void *buf, size_t size)
{
```

```
    void   *addr   = buf;
    size_t  qwords = size >> 3;

    asm volatile ("lfence" ::: "memory");
    if (qwords)
        asm volatile ("rep stosq"
                      : "+D"(addr), "+c"(qwords)
                      : "a"(0ULL) : "memory");

    /* CLFLUSHOPT — evict every cache line */
    uintptr_t start = (uintptr_t)buf & ~(CACHE_LINE - 1UL);
    uintptr_t end   = ((uintptr_t)buf + size + CACHE_LINE - 1UL)
                      & ~(CACHE_LINE - 1UL);
    for (uintptr_t ln = start; ln < end; ln += CACHE_LINE)
        asm volatile ("clflushopt (%0)" :: "r"((void *)ln) : "memory");

    asm volatile ("mfence" ::: "memory");
    asm volatile ("lfence" ::: "memory");
}
```

main() — four measurements
Measures each phase and computes how many attacker measurements fit inside the window.

```
/* ① protect_core latency */
printf("[1] tid_protect_core():\n");
/* min / avg / max over 10000 samples */

/* ② zeroize_core latency — AES-256 key (32 bytes) */
printf("[2] tid_zeroize_core() on 32 bytes (AES-256):\n");

/* ③ attacker speed */
printf("[3] Attacker speed (one Flush+Reload measurement):\n");
uint64_t attacker_avg = 112;    /* ns per measurement */

/* ④ full window */
printf("[4] Full window (data entry -> end of wipe):\n");
printf("    Attacker measurements inside window: %.1f\n",
       (double)avg / attacker_avg);
```

Expected output: Window ≈ 372 ns | Attacker speed ≈ 112 ns/meas | ≈ 3.3 measurements fit

File 3 Race Window Solution — TID Shield — tid_shield.c

Implements three protection layers to shrink the race window, combined in one shield_execute() pattern.

Layer 1 — CPU Isolation (CPU Affinity)
Pins the process to a dedicated core — attacker cannot share the same L1 Cache.

```c
/* ── Layer 1: CPU Isolation ── */
static int set_cpu_affinity(int core)
{
    cpu_set_t mask;
    CPU_ZERO(&mask); CPU_SET(core, &mask);
    return sched_setaffinity(0, sizeof(mask), &mask);
}

static void restore_cpu_affinity(void)
{
    cpu_set_t mask;
    int n = sysconf(_SC_NPROCESSORS_ONLN);
    CPU_ZERO(&mask);
    for (int i = 0; i < n; i++) CPU_SET(i, &mask);
    sched_setaffinity(0, sizeof(mask), &mask);
}
```

Layer 2 — Real-Time Priority (SCHED_FIFO)

Prevents Context Switch during the sensitive window. Requires root privileges.

```c
/* ── Layer 2: Real-Time Priority ── */
static int set_realtime(void)
{
    struct sched_param rt = { .sched_priority = 1 };
    if (sched_setscheduler(0, SCHED_FIFO, &rt) == 0) return 0;
    if (sched_setscheduler(0, SCHED_RR,   &rt) == 0) return 0;
    setpriority(PRIO_PROCESS, 0, -20);
    return -1;
}
```

Layer 3 — Memory Locking (mlock)

Prevents the sensitive buffer from being copied to swap — no hidden copy on disk.

```c
/* ── Layer 3: Memory Locking ── */
static int lock_memory(void *buf, size_t size)
{
    return mlock(buf, size);
}
```

shield_execute()

Combines all three layers in a single protected execution pattern — the sensitive window is fully guarded.

```c
/* ── Full Protected Pattern ── */
static void shield_execute(void *buf, size_t size)
{
    int core     = sysconf(_SC_NPROCESSORS_ONLN) - 1;
    int aff_ok   = set_cpu_affinity(core);
    int mlock_ok = lock_memory(buf, size);
    int rt_ok    = set_realtime();

    /* == SENSITIVE WINDOW == */
    protect_core();
```

```
    /* [ application uses data here ] */
    volatile uint64_t chk = 0;
    uint8_t *p = (uint8_t *)buf;
    for (size_t i = 0; i < size; i++) chk += p[i];
    (void)chk;
    zeroize_core(buf, size);
    /* ═══════════════════════ */

    if (rt_ok    == 0) restore_sched();
    if (mlock_ok == 0) unlock_memory(buf, size);
    if (aff_ok   == 0) restore_cpu_affinity();
}
```

⚙ Build & Run Commands

```
# Build all files
gcc -O2 -Wall -Wextra -mclflushopt -march=native \
    -o tid_attack_test   tid_attack_test.c

gcc -O2 -Wall -Wextra -mclflushopt -march=native \
    -o tid_race_analysis tid_race_analysis.c

gcc -O2 -Wall -Wextra -mclflushopt -march=native \
    -o tid_shield        tid_shield.c

# Run in order:
./tid_attack_test        # Core Flush+Reload attack test
./tid_race_analysis      # Timing gap analysis
./tid_shield             # Race window shield test
```

Proven result: TID v2.0 raises attacker latency 3.7x (78 → 286 cycles) — Flush+Reload defeated after data use ends.

— End of Code —